Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Recherche d'éléments dans une structure linéaire et analyse de complexité temporelle

Synthèse exécutive : Ce cours traite des méthodes fondamentales pour retrouver une valeur dans une liste ou un tableau. L'enjeu central repose sur la relation entre la structure des données (tableau trié ou non trié) et la performance des algorithmes de recherche. La recherche séquentielle parcourt exhaustivement le tableau en complexité linéaire O(n), sans aucune contrainte d'organisation préalable. À l'inverse, la recherche dichotomique tire parti d'un tableau préalablement trié pour appliquer le paradigme 'diviser pour régner', ramenant la complexité à une échelle logarithmique O(log n). L'étudiant doit savoir choisir l'algorithme adapté selon le contexte et exécuter pas à pas la mise à jour des pointeurs d'indices.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est la stratégie la plus intuitive et universelle. Elle consiste à lire un tableau case par case, de l'indice 0 jusqu'à l'indice n-1, et à comparer chaque valeur à l'élément recherché. L'algorithme s'arrête dès que l'élément est trouvé (succès) ou lorsque l'intégralité du tableau a été parcourue sans succès. Si le tableau comporte n éléments, le pire des cas (élément situé tout à la fin ou absent) nécessite n comparaisons. Sa complexité temporelle est donc qualifiée de linéaire, notée O(n). Son grand avantage est d'être applicable à n'importe quel tableau, trié ou non.

- Parcours exhaustif élément par élément du début vers la fin
- Complexité temporelle linéaire O(n) au pire des cas
- Aucune précondition requise sur l'organisation du tableau

Cas concret enseignant : *Le professeur a utilisé l'analogie de la recherche d'une clé tombée dans un tas de feuilles mortes : le tas n'ayant aucune organisation logique, il est impératif de soulever chaque feuille une à une jusqu'à trouver la clé.*

2. La Recherche Dichotomique (Diviser pour Régner)

La recherche dichotomique repose sur le principe algorithmique 'diviser pour régner'. Elle nécessite impérativement que le tableau soit trié au préalable. Au lieu de parcourir le tableau élément par élément, l'algorithme calcule l'indice du milieu : milieu = (debut + fin) // 2. Il compare ensuite l'élément recherché avec la valeur située à l'indice milieu. Si la cible est égale à cette valeur, la recherche est terminée. Si elle est inférieure, la cible ne peut se trouver que dans la première moitié : on déplace le pointeur 'fin' à 'milieu - 1'. Si elle est supérieure, on déplace le pointeur 'debut' à 'milieu + 1'. En éliminant 50 % des candidats restants à chaque étape, l'espace de recherche se réduit très rapidement, garantissant une complexité logarithmique O(log n).

- Précondition obligatoire : Le tableau doit être strictement trié
- Complexité temporelle logarithmique O(log2 n) au pire des cas
- Gestion rigoureuse de trois pointeurs : debut, fin, milieu

Cas concret enseignant : *Le professeur a illustré le concept avec le jeu du 'Devine un nombre entre 1 et 100'. Proposer 1, 2, 3... est une approche séquentielle inefficace. Commencer par 50 permet d'éliminer immédiatement 50 possibilités si la réponse est 'c'est plus grand', assurant la victoire en 7 étapes maximum (log2(100) ≈ 6.64).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est beaucoup plus rapide ?
Réponse du professeur : Parce que la recherche dichotomique exige absolument un tableau trié. Le tri d'un tableau non ordonné coûte du temps de calcul (au mieux O(n log n)). Si vous n'avez qu'une seule recherche à effectuer dans un tableau non trié, trier le tableau puis faire une dichotomie coûtera plus cher que de faire directement une recherche séquentielle en O(n).

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière (notée '//' en Python ou la division d'entiers en Java/C). La partie décimale est tronquée, ce qui pointe automatiquement vers l'élément situé juste avant le centre théorique. Cela conserve un indice valide sans créer d'erreur.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61]. Appliquer pas à pas l'algorithme de recherche dichotomique pour rechercher la valeur 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : Calcul du milieu -> milieu = (0 + 9) // 2 = 4. T[4] = 19. Comparaison : 42 > 19. La valeur est dans la moitié droite. Mise à jour : debut = milieu + 1 = 5. Étape 2 : debut = 5, fin = 9. Calcul du milieu -> milieu = (5 + 9) // 2 = 7. T[7] = 42. Comparaison : T[7] == 42. L'élément est trouvé !

Résultat : La valeur 42 a été localisée à l'indice 7 en seulement 2 comparaisons, alors qu'une recherche séquentielle aurait nécessité 8 comparaisons.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative indispensable pour appliquer une recherche dichotomique ?

R : Le tableau doit impérativement être trié (en ordre croissant ou décroissant).

Q : Quelle est la complexité dans le pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : La complexité est linéaire : O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour un tableau trié de 1024 éléments ?

R : Elle nécessite au maximum 11 comparaisons, car 2^10 = 1024 (soit log2(1024) + 1).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765